

io_uring是Linux内核自5.1版本引入的革命性异步I/O框架，通过共享内存环形队列和无锁并发设计，解决了传统异步I/O模型的局限性，实现了零拷贝、低延迟的高性能I/O处理机制。本文将从设计原理、API实现、性能优势及现代应用实践四个维度，全面剖析这一现代Linux异步I/O技术的革新与价值。

一、设计原理与核心机制

1. 环形缓冲区架构

io_uring的核心设计是围绕两个共享内存环形缓冲区构建的：**提交队列(SQ, Submission Queue)**和**完成队列(CQ, Completion Queue)**。这种设计彻底改变了传统系统调用中内核与用户态之间的通信方式：

- **提交队列(SQ)**：用户态程序将I/O请求封装为SQE(Submission Queue Entry)结构，写入SQ的尾部。SQ采用环形结构，具有head和tail两个指针，分别由用户态和内核态管理。用户态作为生产者，内核作为消费者。
- **完成队列(CQ)**：内核处理完I/O请求后，将结果封装为CQE(Completion Queue Entry)结构，写入CQ的尾部。CQ同样采用环形结构，具有head和tail指针，但此时内核是生产者，用户态是消费者。

```
struct io_uring_sqe {
    __u8  opcode;    /* 操作码，标识I/O类型 */
    __u8  flags;     /* IOSQE_标志，控制请求行为 */
    __u16 ioprio;    /* I/O优先级 */
    __s32 fd;        /* 操作的文件描述符 */
    union {
        __u64 off;    /* 文件偏移量 */
        __u64 addr2;  /* 备用地址字段 */
        struct {
            __u32 cmd_op; /* 命令操作码 */
            __u32 __pad1;
        };
    };
    union {
        __u64 addr;    /* 缓冲区地址或iovec数组地址 */
        __u64 splice_off_in; /* splice操作的输入偏移量 */
    };
    __u32 len;        /* 缓冲区长度或iovec数组元素数 */
    /* ... 其他字段 ... */
};
```

2. 无锁并发模型

io_uring采用**无锁设计**，通过原子操作和内存屏障实现高效并发：

- **原子操作**：使用原子操作(如fetch_add)更新SQ和CQ的头尾指针，避免了传统锁机制带来的竞争开销。
- **内存屏障**：通过mb()等内存屏障指令确保指针修改的可见性，防止指令重排导致的队列状态不一致。
- **单生产者/单消费者模式**：每个SQ和CQ采用单生产者单消费者模式，避免了多线程环境下的复杂同步问题。

内核与用户态通过****内存映射(mmap)****共享这两对环形缓冲区，避免了数据在用户态和内核态之间的拷贝，实现了零拷贝通信。

3. 多模式工作机制

io_uring提供了多种工作模式以适应不同应用场景：

- **标准模式**：用户态通过io_uring_enter系统调用通知内核处理SQ中的请求，并等待CQ中的完成事件。
- **SQPOLL模式**：启用内核轮询线程自动处理SQ中的请求，无需用户态显式调用io_uring_enter提交请求，进一步减少系统调用开销。
- **IOPOLL模式**：内核绕过中断机制，直接轮询硬件设备状态寄存器，以检查I/O是否完成，减少中断处理开销，显著降低延迟。
- **FAST POLL模式**：允许在文件描述符不可读写时，内核主动添加poll handler，当文件描述符可读写时自动调用poll handler再次下发读写请求，特别适合网络操作。

io_uring的创新在于其统一的异步I/O接口设计，几乎支持所有系统调用，包括read、write、accept、connect、fsync、open、sendmsg、recvmsg等，而传统Linux AIO仅支持有限的存储文件操作。

二、API与系统调用解析

1. 核心系统调用

io_uring仅提供三个核心系统调用，但功能强大：

- **io_uring_setup**：初始化io_uring实例，创建SQ和CQ，返回文件描述符。

```
int io_uring_setup (u32 entries, struct io_uring_params *params);
```

参数entries指定SQ的最小长度，params返回SQ、CQ和SQEs的配置信息。通过mmap将内核创建的队列地址映射到用户空间，实现共享内存。

- **io_uring_enter**：提交I/O请求并等待完成事件。

```
int io_uring_enter (unsigned int fd, unsigned int to_submit,
                   unsigned int min_complete, unsigned int flags,
                   sigset_t *sig);
```

参数to submit指定要提交的SQE数量，min complete指定要等待的最小完成事件数，flags控制行为，如IORING Enter getevents表示等待完成事件，IORING Enter wait表示阻塞等待。

- **io_uring_register**：注册用户态和内核态共享的缓冲区或文件描述符。

```
int io_uring_register (unsigned int fd, unsigned int opcode,
                      void *arg, unsigned int nr_args);
```

2. liburing库封装

为简化使用，Linux社区提供了liburing库封装：

- **初始化与销毁**：

```
int io_uring_queue_init(unsigned entries, struct io_uring *ring,
                       unsigned flags);
void io_uring_queue_init_params(unsigned entries, struct io_uring *ring,
                               struct io_uring_params *params);
void io_uring_queue_exit(struct io_uring *ring);
```

- **请求提交**：

```
struct io_uring_sqe *io_uring_get_sqe(struct io_uring *ring);
int io_uring_submit(struct io_uring *ring);
int io_uring_submit_and_wait(struct io_uring *ring, unsigned min);
```

- **结果等待**：

```
int io_uring_wait_cqe(struct io_uring *ring, struct io_uring_cqe **cqe);
int io_uring_peek_batch_cqe(struct io_uring *ring, struct io_uring_cqe **cqes,
                           unsigned nr);
void io_uring_cqadvance(struct io_uring *ring);
```

liburing通过io_uring_queue_init封装了io_uring_setup和mmap，简化了队列的初始化过程。

io_uring_get_sqe获取空闲的SQE，io_uring submit通过原子操作更新SQ尾指针提交请求，而

io_uring_wait_cqe则通过io_uring_enter的IORINGEnter getevents标志实现完成事件的等待。

3. 提交队列项(SQE)配置

SQE是io_uring的请求描述结构，通过不同的准备函数配置：

// 文件I/O操作

```
io_uring prep_read(struct io_uring_sqe *sqe, int fd, void *buf,  
                  off_t off, size_t len);
```

```
io_uring prep_write(struct io_uring_sqe *sqe, int fd, const void *buf,  
                   off_t off, size_t len);
```

// 网络操作

```
io_uring prep_sendmsg(struct io_uring_sqe *sqe, int fd,  
                     const struct msghdr *msg);
```

```
io_uring prep接受(struct io_uring_sqe *sqe, int fd,  
                  int *newfd, struct sockaddr *addr,  
                  socklen_t *addrlen);
```

每个SQE包含操作码opcode、文件描述符fd、缓冲区地址addr和长度len等关键字段，用户态填充这些字段后，通过提交队列通知内核执行。

三、性能优势与测试数据

1. 系统开销对比

与传统异步I/O模型相比，io_uring在系统开销上有显著优势：

- **内存拷贝开销**：传统AIO每次提交需要拷贝64+8字节，每次完成需要拷贝32字节，总计104字节。而io_uring通过共享内存完全消除了这一开销。
- **系统调用次数**：传统AIO需要两次系统调用才能完成一个I/O操作（submit和wait-for-completion），而io_uring通过io_uring_enter系统调用实现提交和等待的原子操作，大大减少了系统调用次数。
- **锁竞争**：io_uring的环形缓冲区采用无锁设计，通过内存屏障减少了竞争，而传统模型通常需要锁来同步队列访问。

2. 实际性能测试数据

根据多个实测数据，io_uring在不同场景下的性能表现如下：

存储I/O场景：

- FIO测试显示，4K随机读写操作中，io_uring的IOPS可达19.0K，是同步模式(psync)的2倍以上。
- 随着I/O深度(I/O depth)增加，io_uring的性能优势更加明显。例如，随机读操作中，当I/O深度从1增加到8时，io_uring的IOPS从139K提升到512K，而传统AIO的提升幅度较小。

网络I/O场景：

- 在Nginx长连接测试中，io_uring相比epoll有20%+的QPS提升，系统调用减少90%以上，尤其在高并发连接数(>500)时优势显著。
- 然而，在短连接场景中，io_uring的性能可能下降5%-10%，主要原因是批量提交引入额外延迟或请求失败的资源开销。

数据库I/O场景：

- 在RocksDB的MultiGet测试中，通过io_uring的异步读操作，性能提升显著，但写操作因未适配仍受限于传统模型。
- 与传统AIO相比，io_uring通常能达到两倍于AIO的性能，特别是在高并发和大块数据I/O场景下。

系统调用优化：

- 在Nginx的实现中，基于io_uring的编程模型几乎消除了对epoll_wait的依赖，系统调用次数大幅减少，显著提高了CPU利用率。
- 通过io_uring_enter的参数组合，可以实现批量提交和批量收割完成事件，进一步提高了吞吐量。

四、现代应用实践

1. C++协程与io_uring结合

C++20引入了协程支持，与io_uring结合可实现高性能异步运行时：

- **协程挂起机制**：通过co_await挂起协程，并将协程上下文与SQE的user_data字段关联。当CQE完成时，协程自动恢复执行。
- **源码实现**：在zedio项目中，协程通过io_uring submit提交请求，然后通过io_uring enter的IORINGEnter getevents标志收割完成事件，实现真正的非阻塞I/O。
- **性能优势**：在echo_server基准测试中，C++协程+io_uring实现的异步运行时性能与Tokio持平，证明了其在高性能网络应用中的潜力。

// 示例：协程中使用io_uring提交读请求

腐蚀协程

```
async_file_read(int fd, void* buf, off_t off, size_t len) {
    // 获取SQE
    struct io_uring_sqe* sqe = io_uring_get_sqe(&ring);
    // 准备读操作
    io_uring_prep_read(sqe, fd, buf, off, len);
    // 提交请求
    io_uring_submit(&ring);
    // 等待完成
    struct io_uring_cqe* cqe;
    while (1) {
        // 提取完成事件
        int ret = io_uring_wait_cqe(&ring, &cqe);
        // 检查是否是当前请求
        if (cqe->user_data == current_request_id) {
            // 处理结果
            break;
        }
    }
    // 标记完成事件已处理
    io_uring_cqadvance(&ring);
}
```

2. Rust Tokio框架与io_uring集成

Tokio作为Rust的异步运行时，已通过tokio-uring crate提供io_uring支持：

- **集成方式**：tokio-uring封装了底层io_uring系统调用，提供了与Tokio原生API兼容的异步文件/网络操作。
- **性能表现**：在特定场景下，如TCP echo服务器，性能提升可达60%。然而，在高连接数场景下，性能提升可能有限，甚至可能出现延迟增加的情况。
- **最佳实践**：Tokio建议使用IORING Setup defer taskrun和IORING Setup taskrun flag标志来优化任务调度，避免不必要的上下文切换。

// 示例：Rust中使用tokio-uring读取文件

```
let file = File::open("test.txt").await?;
```

```
let mut buf = vec![0; 1024];
```

// 提交读请求

```
file.read_at(&mut buf, 0).await?;
```

// 处理结果

3. 高性能网络服务器应用

io_uring在网络服务器中的应用主要体现在以下几个方面：

- **FAST POLL机制**：允许在文件描述符不可读写时直接提交请求，内核会自动处理poll事件，无需额外注册。
- **批量处理**：支持批量提交和批量收割完成事件，特别适合高并发连接场景。
- **零拷贝传输**：在网络操作中，通过注册缓冲区，可以实现零拷贝数据传输，显著提高吞吐量。

在Nginx的io_uring优化中，通过io_uring_enter系统调用替代传统的epoll_wait，减少了系统调用次数，提高了CPU利用率。实测显示，在长连接场景下，QPS提升20%以上，系统调用减少90%。

4. 存储系统与数据库应用

io_uring在存储系统和数据库应用中也有广泛应用：

- **文件系统操作**：支持buffered I/O和direct I/O，充分利用文件系统缓存，减少数据碰盘产生的系统延迟。
- **数据库优化**：RocksDB已尝试使用io_uring加速其MultiGet功能，通过异步读取多个SST文件，显著提高了查询性能。
- **虚拟化环境**：在Quark容器运行时中，通过io_uring直接与宿主机内核交互，绕过虚拟化层的QCall机制，进一步加速I/O操作。

io_uring的适用场景：适合I/O密集型应用，如高并发Web服务器、分布式存储、数据库读操作、NVMe SSD驱动等。

io_uring的局限性：在短连接场景中性能提升有限，甚至可能下降；对小数据量操作(如8byte)的优化效果有限；MySQL等传统数据库尚未原生支持io_uring，需通过第三方补丁或文件系统级优化。

五、实现机制与优化策略

1. 队列管理实现

io_uring的队列管理实现是其高性能的关键：

- **环形缓冲区结构**：SQ和CQ采用环形结构，通过head和tail指针管理，head=tail表示队列为空。
- **SQEs数组**：额外提供一个SQEs数组，保存实际的I/O请求。SQ和CQ中的每个节点保存的是SQEs数组的偏移量，而非实际请求，这样可以批量提交内存上不连续的请求。

- **mmap映射**：通过io_uring_setup获取文件描述符后，使用mmap将内核创建的队列地址映射到用户空间，实现共享内存。

// 示例：通过mmap映射环形缓冲区

```
struct io_uring_params params;
io_uring_setup(entries, &params);
void *sq ring = mmap(NULL, params.sq ring sz, PROT_READ | PROT_WRITE,
    MAP_SHARED | MAP Populate, params ring fd, params.sq ring off);
void *cq ring = mmap(NULL, params.cq ring sz, PROT_READ | PROT_WRITE,
    MAP_SHARED | MAP Populate, params ring fd, params.cq ring off);
```

2. 无锁同步机制

io_uring的无锁同步机制是其实现高效并发的关键：

- **原子操作**：使用fetch_add等原子操作更新SQ和CQ的头尾指针，确保线程安全性。
- **内存屏障**：通过mb()等内存屏障指令确保指针修改的可见性，防止指令重排导致的队列状态不一致。
- **用户态与内核态协作**：用户态通过更新SQ的tail指针提交请求，内核通过更新SQ的head指针标记已处理请求；内核通过更新CQ的tail指针提交完成事件，用户态通过更新CQ的head指针标记已处理完成事件。

3. 高级优化策略

io_uring提供了多种高级优化策略：

- **SQPOLL模式**：启用内核轮询线程自动处理SQ中的请求，无需用户态显式调用io_uring_enter提交请求，进一步减少系统调用开销。
- **IOPOLL模式**：内核绕过中断机制，直接轮询硬件设备状态寄存器，减少中断处理开销，降低延迟。
- **FAST POLL模式**：允许在文件描述符不可读写时，内核主动添加poll handler，当文件描述符可读写时自动调用poll handler再次下发读写请求，特别适合网络操作。
- **环间消息(Ring messages)**：允许在两个io_uring实例之间传递消息，可以用于唤醒阻塞的io_uring或在不同线程间传递数据。

// 示例：使用FAST POLL模式提交网络请求

```
struct msghdr msg;
// 准备msghdr
io_uring_prep_sendmsg(sqe, fd, &msg);
```



```
// 设置FAST POLL标志
sqe->flags |= IORING OP sendmsg | IORING sendmsg poll first;
// 提交请求
io_uring submit(&ring);
```

六、未来发展趋势与挑战

1. 技术演进方向

io_uring技术正在快速发展，未来可能向以下方向演进：

- **更广泛的操作支持**：目前io_uring已支持多种系统调用，但仍有扩展空间，未来可能支持更多类型的I/O操作。
- **更高效的轮询机制**：如IORING Setup defer taskrun和IORING Setup taskrun flag等标志的引入，优化了任务调度，减少不必要的上下文切换。
- **更好的兼容性**：随着Linux内核版本的更新，io_uring的兼容性将不断提升，支持更多硬件和文件系统。

2. 集成挑战与解决方案

io_uring的集成面临一些挑战：

- **资源管理**：需要合理设置队列深度，根据实际负载调整，避免队列溢出。
- **错误处理**：需要完善错误处理机制，特别是对于accept()等操作，避免无效内存申请和释放。
- **性能调优**：不同工作负载下，io_uring的性能表现可能差异较大，需要针对性调优。

解决方案：

- 合理设置队列深度，根据系统资源和工作负载进行调整。
- 使用io_uring prep msg ring()等函数处理环间消息，优化任务调度。
- 结合具体应用场景，选择合适的io_uring模式和参数组合，如长连接使用FAST POLL模式，短连接考虑其他优化策略。

七、结论与建议

io_uring作为Linux内核的革命性异步I/O框架，通过共享内存环形队列和无锁并发设计，解决了传统异步I/O模型的局限性，实现了零拷贝、低延迟的高性能I/O处理机制。其核心优势在于：

1. **零拷贝通信**：通过共享内存环形队列，避免了用户态和内核态之间的数据拷贝。

2. **减少系统调用**：通过io_uring_enter系统调用实现提交和等待的原子操作，大大减少了系统调用次数。
3. **无锁并发模型**：采用原子操作和内存屏障实现无锁访问，减少了锁竞争，提高了并发性能。
4. **广泛的操作支持**：几乎支持所有系统调用，包括文件I/O和网络操作，而传统Linux AIO仅支持有限的存储文件操作。
5. **多种工作模式**：支持标准模式、SQPOLLMODE、IOPOLL模式和FAST POLL模式等多种工作模式，适应不同应用场景。

对于开发者，建议根据具体应用场景选择是否采用io_uring：

- **推荐场景**：高并发长连接应用（如Web服务器）、大块数据I/O、NVMe SSD等高性能存储设备驱动的数据库读操作。
- **谨慎场景**：短连接网络服务、小数据量操作、需要严格控制I/O顺序的应用。
- **实施建议**：从简单I/O操作开始集成io_uring，逐步扩展到复杂场景；合理设置队列深度；针对不同操作类型选择合适的io_uring模式；结合协程等现代编程模型，充分发挥io_uring的性能优势。

随着Linux内核版本的更新和应用生态的成熟，io_uring有望成为Linux异步I/O的标准接口，为更多高性能应用提供支持。

参考来源

[1]存储大师班|LinuxIO模式之io_uring-InfoQ 写作平台

<https://xie.infoq.cn/article/9b07b729101d7abcd5db101ac>

[2]io_uring系统调用及示例-CSDN博客

<https://blog.csdn.net/zidier215/article/details/149947808>

[3]异步IO(io_uring)性能测试-CSDN博客

https://blog.csdn.net/qj_33248019/article/details/154600099

[4]Linux之IO_URING-CSDN博客

<https://blog.csdn.net/yelede2009/article/details/152231079>

[5][学习笔记]2_5-io_uring-CSDN博客

https://blog.csdn.net/2503_92663309/article/details/151937343

[6]io_uring异步IO框架介绍与示例-pureflash-博客园

<https://www.cnblogs.com/pureflash/p/18731118>

[7]【系统级编程进阶】C++中基于io_uring的极致性能优化实践-CSDN博客

<https://blog.csdn.net/compitide/article/details/155161508>

[8]关于io_uring-Tohomson-博客园

<https://www.cnblogs.com/Tohomson/p/18802775>

[9]io_uring最简单的实例io_uring-test.c分析-CSDN博客

<https://blog.csdn.net/zfxwasaboy/article/details/151764887>

[10]转：[译]Linux 异步 I/O 框架io_uring：基本原理、程序示例与性能压测（2020）mohist-博客园

<https://www.cnblogs.com/pandamohist/p/17603530.html>

[11]【现代C++】基于C++协程与IO-uring实现的异步运行时项目。（使用与性能测试）

<https://www.bilibili.com/video/BV1de411h71r>

[12]Quark: A High-Performance Secure Container Runtime for Serverless Computing

<https://arxiv.org/abs/2309.12624>

[13]io_uring-guanyubo-博客园

<https://www.cnblogs.com/yubo-guan/p/18104456>

[14]Reducing Memory Contention and I/O Congestion for Disk-based GNN Training

<https://arxiv.org/abs/2406.13984>

[15]AisLSM: Revolutionizing the Cooutput with Asynchronous I/Os for Compaction with LM-tree

<https://arxiv.org/abs/2307.16693>

[16]High-performance and Scalable Software-based NVMeVirtualization Mechanism with I/O Queues Passthrough

<https://arxiv.org/abs/2304.05148>

[17]存储大师班|LinuxIO模式之io_uring-InfoQ 写作平台

<https://xie.infoq.cn/article/9b07b729101d7abcd5db101ac>

[18]CAPSTONE: A Capability-based Foundation for Trustless Secure Memory Access (Extended Version)

<https://arxiv.org/abs/2302.13863>

[19]Reading from External Memory

<https://arxiv.org/abs/2102.11198>

[20]Reducing Memory Contention and I/O Congestion for Disk-based GNN Training

<https://arxiv.org/abs/2406.13984>

[21]io_uring_enter：io_uring 框架的 I/O 提交与事件等待接口-CSDN博客

https://blog.csdn.net/m0_51729057/article/details/149720529

[22]Performance Evaluation and Modeling of HPC I/O on Non-Volatile Memory

<https://arxiv.org/abs/1705.03598>

[23]面试官:io_uring相比LinuxAIO性能提高5%,为什么成为革命性技术? 腾讯云开发者社区-腾讯云

<https://cloud.tencent.com/developer/article/2531325>

- [24]Quark: A High-Performance Secure Container Runtime for Serverless Computing
<https://arxiv.org/abs/2309.12624>
- [25]An Adaptive Parallel Arc-Length Method
<https://arxiv.org/abs/2303.01075>
- [26]内核飞跃篇：io_uring与真正的异步 I/O-CSDN博客
https://blog.csdn.net/qq_43727159/article/details/151937497
- [27]AisLSM: Revolutionizing the Cooutput with Asynchronous I/Os for Compaction with LM-tree
<https://arxiv.org/abs/2307.16693>
- [28]io_uring概述：现代 Linux 异步 IO 的新范式_linuxiouring-CSDN博客
https://blog.csdn.net/2401_84703565/article/details/147542399
- [29]Bringing Reconfigurability to the Network Stack
<https://arxiv.org/abs/2311.07753>
- [30]深入理解liburing核心API：从提交队列到完成队列的完整指南-CSDN博客
https://blog.csdn.net/gitblog_00898/article/details/145363609
- [31]探索Linux下高性能异步I/O：用liburing实现的Echo服务器-CSDN...
https://blog.csdn.net/weixin_45634782/article/details/138038384
- [32]io_uring_enter：io_uring 框架的 I/O 提交与事件等待接口-CSDN博客
https://blog.csdn.net/m0_51729057/article/details/149720529
- [33]Quark: A High-Performance Secure Container Runtime for Serverless Computing
<https://arxiv.org/abs/2309.12624>
- [34]Reducing Memory Contention and I/O Congestion for Disk-based GNN Training
<https://arxiv.org/abs/2406.13984>
- [35]一文详细讲解io_uring-哔哩哔哩
<https://www.bilibili.com/read/cv19693879/>
- [36]AisLSM: Revolutionizing the Cooutput with Asynchronous I/Os for Compaction with LM-tree
<https://arxiv.org/abs/2307.16693>
- [37]How to use Persistent Memory in your Database
<https://arxiv.org/abs/2112.00425>
- [38]Nginx io_uring优化对长连接性能的提升效果与原理分析-开发者...
<https://developer.aliyun.com/article/774857>
- [39]Quark: A High-Performance Secure Container Runtime for Serverless Computing
<https://arxiv.org/abs/2309.12624>
- [40]MySQL异步I/O性能优化全解析-CSDN博客
https://blog.csdn.net/m0_37607945/article/details/152085209
- [41]io_uring优化nginx实战演练-今日头条

<https://www.toutiao.com/article/7161692691258950148/>

[42]Reducing Memory Contention and I/O Congestion for Disk-based GNN Training

<https://arxiv.org/abs/2406.13984>

[43]异步I/O子系统

<https://www.ibm.com/docs/zh/aix/7.2.0?topic=concepts-asynchronous-io-subsystem>

[44]面对疾风吧！io_uring优化nginx实战演练_openanolis-CSDN博客

<https://blog.csdn.net/openanolis/article/details/109745598>

[45]AisLSM: Revolutionizing the Cooutput with Asynchronous I/Os for Compaction with LM-tree

<https://arxiv.org/abs/2307.16693>

[46]MySQL-异步I/O-abce-博客园

<https://www.cnblogs.com/abclife/p/7639735.html>

[47]面对疾风吧！io_uring 优化 nginx 实战演练-CSDN博客

<https://blog.csdn.net/21cnbao/article/details/108878322>

[48]Quark: A High-Performance Secure Container Runtime for Serverless Computing

<https://arxiv.org/abs/2309.12624>

[49]Reducing Memory Contention and I/O Congestion for Disk-based GNN Training

<https://arxiv.org/abs/2406.13984>

[50]BPF for storage: an exokernel-inspired approach

<https://arxiv.org/abs/2102.12922>

[51]异步IO框架，异步io的recv/send/accept，io_uring的性能测试

<https://www.bilibili.com/video/BV1uo4y1b7Yk>

[52]【高性能异步IO机制解析：IO_URING】 | 异步io/io_uring的性能测试/io_uring底层原理/准备linux环境 从手写代码开始

<https://www.bilibili.com/video/BV12Y41167hu>

[53]Reducing Memory Contention and I/O Congestion for Disk-based GNN Training

<https://arxiv.org/abs/2406.13984>

[54]Implementing and Evaluating E2LSH on Storage

<https://arxiv.org/abs/2403.16404>

[55]A Cyber Manufacturing IoT System for Adaptive Manufacturing Model Deployment by Interactive Causality Enabled Self-Labeling

<https://arxiv.org/abs/2404.05976>

[56]Quark: A High-Performance Secure Container Runtime for Serverless Computing

<https://arxiv.org/abs/2309.12624>

[57]Reducing Memory Contention and I/O Congestion for Disk-based GNN Training

<https://arxiv.org/abs/2406.13984>

[58]Quark: A High-Performance Secure Container Runtime for Serverless Computing

<https://arxiv.org/abs/2309.12624>

[59]Support for Parallel and Concurrent Programming in C++

<https://link.springer.com/content/pdf/10.1134/S0361768818010073.pdf>

[60]Tokio.io-uring使用与安装教程*-CSDN博客

https://blog.csdn.net/gitblog_00592/article/details/141733741

[61]Quark: A High-Performance Secure Container Runtime for Serverless Computing

<https://arxiv.org/abs/2309.12624>

[62]DiskGNN: Bridging I/O Efficiency and Model Accuracy for Out-of-Core GNN Training

<https://arxiv.org/abs/2405.05231>

[63]探索未来I/O : tokio-uring-强大的异步IO库-CSDN博客

https://blog.csdn.net/gitblog_00095/article/details/138944461

[64]Reducing Memory Contention and I/O Congestion for Disk-based GNN Training

<https://arxiv.org/abs/2406.13984>

[65]Libfork: portable continuation coroutines

<https://arxiv.org/abs/2402.18480>

[66]【Rust日报】2021-04-09 tokio 计划发布基于 io-uring 的新...

<https://blog.csdn.net/u012067469/article/details/115562250>

[67]AisLSM: Revolutionizing the Cooutput with Asynchronous I/Os for Compaction with LM-tree

<https://arxiv.org/abs/2307.16693>

[68]【现代C++】基于C++协程与IO-uring实现的异步运行时项目。（使用与性能测试）

<https://www.bilibili.com/video/BV1de411h71r>

[69]Reducing Memory Contention and I/O Congestion for Disk-based GNN Training

<https://arxiv.org/abs/2406.13984>

[70]Rust 世界中主流的异步运行时性能测试Tokio/Tokio-uring/MonoIO/GlommIO-piperck-博客园

<https://www.cnblogs.com/piperck/p/18597976>

[71]Libfork: portable continuation coroutines

<https://arxiv.org/abs/2402.18480>

[72]Mitigating Spectre-PHT using Speculation Barriers in Linux BPF

<https://arxiv.org/abs/2405.00078>

[73]Shared memory parallelism in Modern C++ and HPX

<https://arxiv.org/abs/2302.07191>

[74]高效存储：深入解析io_uring的优势与实现

<https://baijiahao.baidu.com/s?id=1847618640057330992>

[75]Web框架性能终极对决：谁才是真正的速度王者[20260108161343]Github项目推荐-博客园

<https://www.cnblogs.com/ltp/p/19459033>

[76]AESTream: Accelerated event-based processing with coroutines

<https://arxiv.org/abs/2212.10719>

(AI生成)